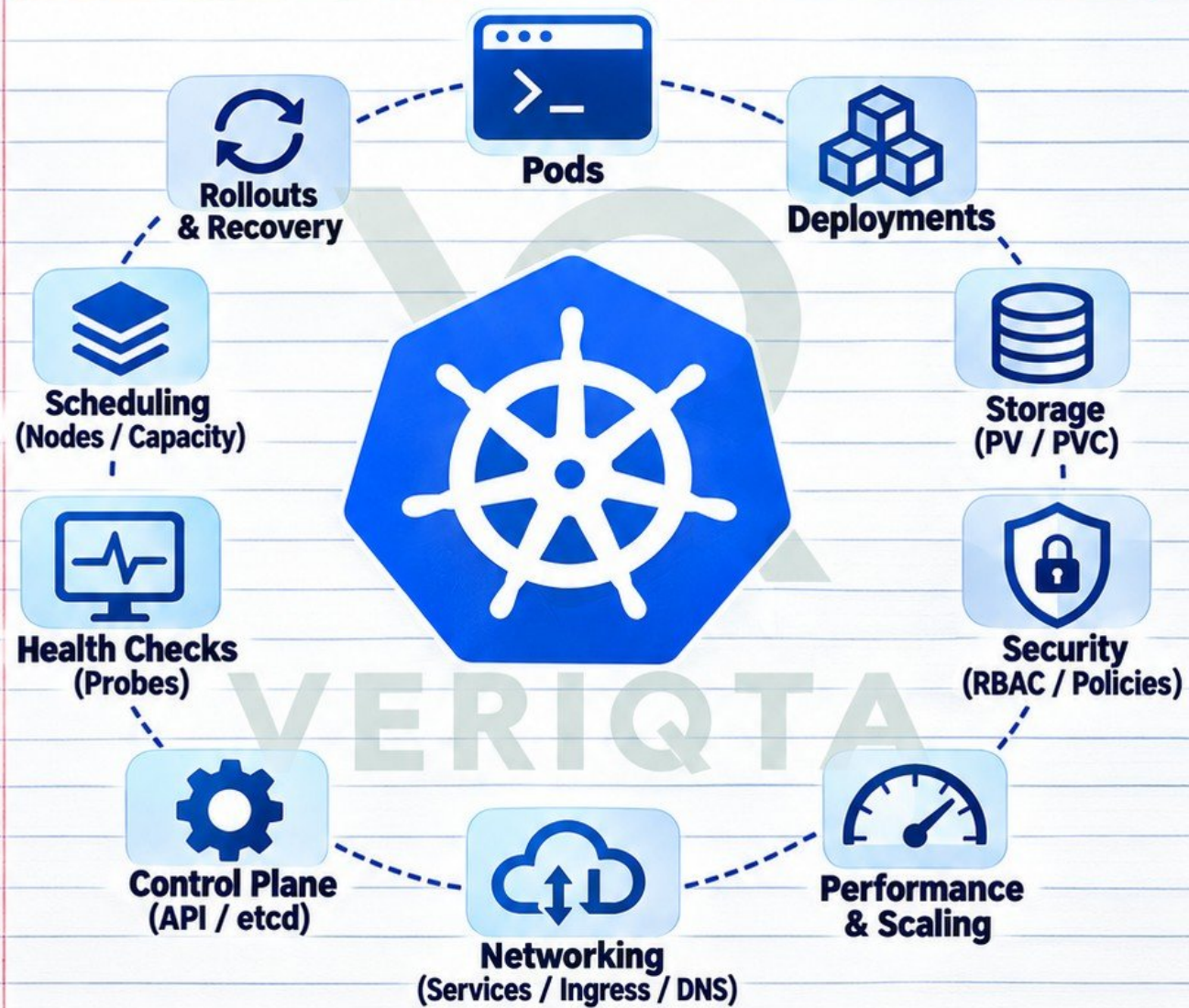
VERIQTA

The Kubernetes Troubleshooting Handbook

50 Errors, Causes, and Fixes



VERIQTA

The Pod Is Running, but the Application Is Dead

Error 01 — Running Pod, Failing Requests

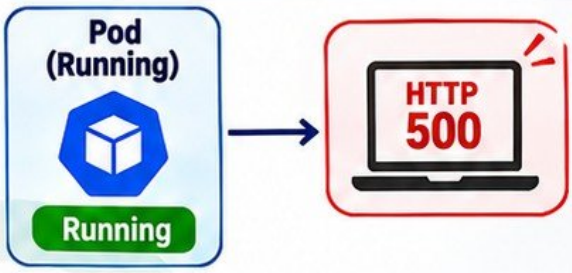
Symptom: Pod phase is Running, but users receive HTTP 500 errors.

Causes: Application deadlock, exhausted worker pool, failed downstream dependency, or incorrect readiness behavior.

Investigate: Compare kubectl get pods, readiness conditions, application logs, and request latency.

Fix: Restore the failing dependency or application execution path. Ensure readiness reflects actual serving capability.

Senior Insight: Kubernetes Pod phase is not an application availability guarantee.




```
$ kubectl get pods -n app
NAME                    READY   STATUS    RESTARTS
api-7f9d9c5b7c-x2kq9    1/1     Running   0
$ kubectl logs api-7f9d9c5b7c-x2kq9 -n app
ERROR: downstream service timeout
ERROR: failed to process request (500)
```

Error 02 — Readiness Probe Passes While Dependencies Are Broken

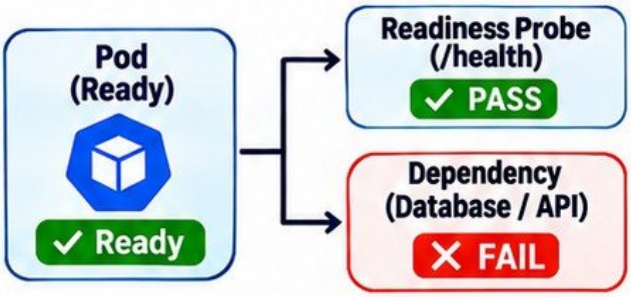
Symptom: The Service continues routing traffic to a Pod that cannot complete business requests.

Causes: A shallow readiness endpoint, incorrect dependency checks, or partial application failure.

Investigate: Compare probe results with representative application transactions and dependency telemetry.

Fix: Define readiness around the application's ability to serve traffic, without making every optional dependency a hard requirement.

Senior Insight: Overly strict readiness checks can turn a partial dependency failure into a full service outage.

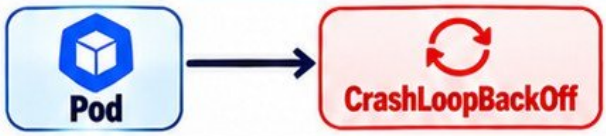



```
$ kubectl describe pod api-6d8f77d9d4-rk8pt
Readiness:  http-get /health  →  Success
Events:
... Readiness probe succeeded
... Warning: downstream dependency unavailable
```

CrashLoopBackOff Is Not the Root Cause

Error 03 — Container Restarts After a Successful Startup

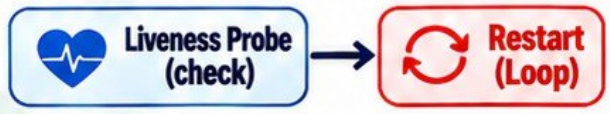
Symptom:	The container starts, runs briefly, then repeatedly exits.
Causes:	Process termination, failed initialization, missing configuration, or application exceptions.
Investigate:	Use kubectl logs --previous, container termination reasons, and exit codes.
Fix:	Correct the actual startup or runtime failure rather than increasing restart delays.
Verification:	The process remains healthy and restart counts stop increasing.




```
$ kubectl get pod
NAME                          READY     STATUS
api-7f9d9c5b7c-xy9k2          0/1       CrashLoopBackOff
$ kubectl logs api-7f9d9c5b7c-xy9k2 --previous
ERROR: missing required environment variable
  CONFIG_PATH
```

Error 04 — Liveness Probe Creates an Endless Restart Loop

Symptom:	A slow-starting application repeatedly restarts before becoming ready.
Causes:	Aggressive liveness thresholds, missing startup probes, or resource starvation.
Investigate:	Correlate probe failures, container startup duration, and CPU throttling.
Fix:	Use a startup probe where appropriate and tune liveness against measured application behavior.
Senior Insight:	A poorly designed health check can become the source of the outage.




```
# Example: Add a startup probe
startupProbe:
    httpGet:
      path: /health
      port: 8080
    initialDelaySeconds: 60
    periodSeconds: 10
    failureThreshold: 6
```

Error 05 — Exit Code 137 Without an Obvious Memory Leak

Symptom:	A container exits with code 137.
Causes:	Possible OOM kill, forced termination after grace-period expiry, or another SIGKILL event.
Investigate:	Check lastState.terminated.reason, container memory metrics, node events, and termination history.
Fix:	Address the confirmed termination cause, such as memory limits or shutdown behavior.
Senior Insight:	Exit code 137 alone does not prove OOMKilled.



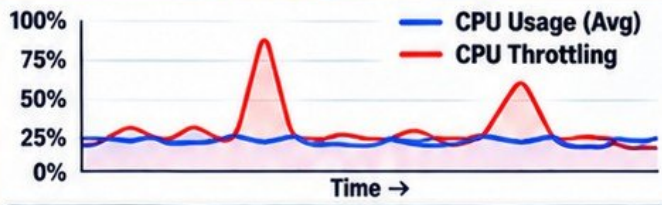

```
$ kubectl describe pod api-7f9d9c5b7c-xy9k2
Last State:
  Terminated:
    Reason: OOMKilled
    Exit Code: 137
    Finished: 2026-11-01T10:24:31Z
```

Resource Limits That Quietly Destroy Performance

Error 06 — CPU Throttling While CPU Utilization Looks Normal

Symptom:	High tail latency despite apparently moderate average CPU usage.
Causes:	CFS quota enforcement, bursty workloads, or poorly chosen CPU limits.
Investigate:	CPU throttling counters, request latency, and container CPU limits.
Fix:	Adjust CPU allocation or limits based on workload measurements.
Senior Insight:	Average CPU usage can hide short periods of severe contention.

CPU Usage vs Throttling




```
$ kubectl top pod api -n app
NAME                CPU(cores)        MEMORY
api-7f9d9c          040               512Mi
$ kubectl exec api -- cat /sys/fs/cgroup/
cpu_stat | grep throttled
nr_throttled        48217
throttled_time      1394827312264
```

Error 07 — Memory Pressure Triggers Unpredictable Pod Evictions

Symptom:	Pods disappear or restart during node memory pressure.
Causes:	Node memory exhaustion, workload requests, QoS behavior, and eviction thresholds.
Investigate:	Node conditions, eviction events, memory working sets, and Pod resource requests.
Fix:	Restore node headroom, correct requests, and rebalance workloads.
Senior Insight:	A Pod's memory limit and its eviction priority are different mechanisms.

Kubernetes Eviction Flow




```
$ kubectl describe node worker-1 | grep -i pressure
Conditions:
  Type:     MemoryPressure
  Status:   True
Events:
  Warning   Evicting pod api-7f9d9c: node low on memory
```

Error 08 — Ephemeral Storage Exhaustion Disrupts Healthy Applications

Symptom:	Pods are evicted despite healthy CPU and memory metrics.
Causes:	Container logs, writable layers, temporary files, or emptyDir usage.
Investigate:	DiskPressure, eviction events, ephemeral storage usage, and node filesystem capacity.
Fix:	Control temporary storage growth and set appropriate storage requests and limits.
Prevention:	Monitor node filesystem and workload ephemeral storage consumption.

Common Ephemeral Storage Sources




```
$ kubectl describe node worker-1 | grep -i disk
Conditions:
  Type:     DiskPressure
  Status:   True
Events:
  Warning   Evicted pod web-6c8f: ephemeral storage exceeded
```

Pending Pods in a Cluster With Free CPU

Error 09 — Scheduler Rejects Nodes With Available Resources

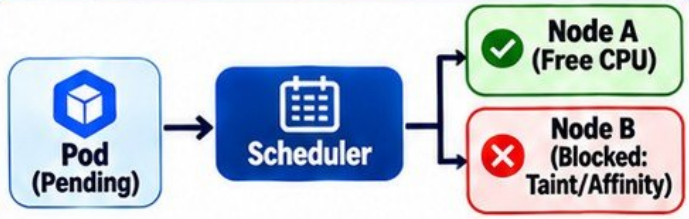
Symptom: Pods remain Pending while monitoring dashboards show spare capacity.

Causes: Insufficient requested resources, affinity constraints, taints, or topology restrictions.

Investigate: Check pod scheduling events, node allocatable capacity, requests, and scheduler constraints.

Fix: Resolve the specific scheduling restriction (requests, taints, affinity, or topology).

Senior Insight: Kubernetes schedules primarily against resource requests, not current CPU usage.




```
$ kubectl describe pod api-7f9d9c5b7c
Events:
  ::: Warning   FailedScheduling
  0/3 nodes are available: 1 node(s) had taint,
  2 node(s) didn't match node affinity.
```

Error 10 — Topology Spread Constraints Block Deployment

Symptom: New replicas remain Pending after an Availability Zone loses capacity.

Causes: Strict topology spread rules, insufficient eligible nodes, or incompatible scheduling constraints.

Investigate: Review topologySpreadConstraints, zone labels, and scheduler events.

Fix: Adjust placement policy or restore zone capacity while preserving required fault isolation.

Trade-off: Strict distribution versus workload availability during partial failure.




```
$ kubectl get deploy web -o yaml | grep -A6
  topologySpreadConstraints
   topologySpreadConstraints:
   - maxSkew: 1
     topologyKey: topology.kubernetes.io/zone
     whenUnsatisfiable: DoNotSchedule
```

Error 11 — Cluster Autoscaler Cannot Add Useful Capacity

Symptom: Pending Pods persist even though autoscaling is enabled.

Causes: Node group limits, incompatible instance types, cloud quotas, or Pod scheduling constraints.

Investigate: Check autoscaler logs, node group capacity, Pod requests, and scheduling events.

Fix: Correct the actual capacity or node-template constraint (e.g., instance type, limits, or quotas).

Senior Insight: Adding nodes is ineffective when the new nodes cannot satisfy the Pod's requirements.




```
$ kubectl -n kube-system logs deploy/
  cluster-autoscaler --tail 5
I1014 10:22:31   ...  Not adding node: node group
  "workers" at max size (5/5)
I1014 10:22:31   ...  Pod requires instance type
  with more memory than available.
```

Node Failures That Become Application Outages

Error 12 — Node NotReady Causes Prolonged Service Disruption

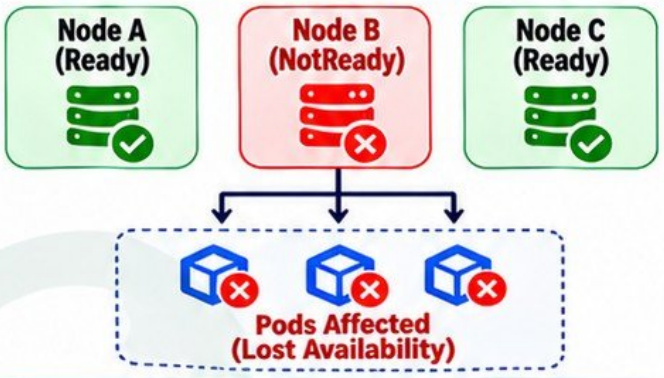
Symptom: A node becomes NotReady and affected workloads lose availability.

Causes: Kubelet failure, runtime problems, network partitions, or resource exhaustion.

Investigate: Check node conditions, kubelet logs, node leases, and workload placement.

Fix: Restore node health or recover workloads onto healthy capacity.

Senior Insight: Node detection, Pod eviction, and application recovery occur on different timelines.




```
$ kubectl get nodes
NAME        STATUS      ROLES      AGE
node-a      Ready       worker     12d
node-b      NotReady    worker     12d
node-c      Ready       worker     12d
$ kubectl describe node node-b
 Conditions:
   Type:      Ready
   Status:    False
   Reason:    KubeletNotReady
```

Error 13 — PodDisruptionBudget Blocks a Critical Node Drain

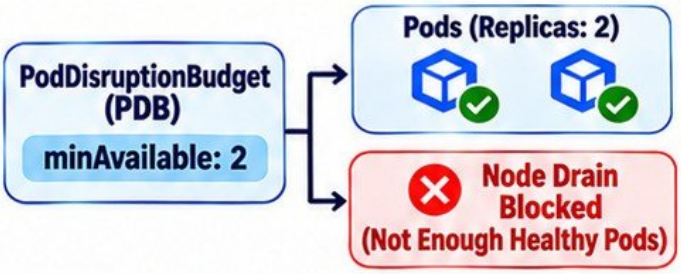
Symptom: Planned maintenance cannot evict certain Pods.

Causes: Insufficient healthy replicas, restrictive disruption budgets, or workloads already unavailable.

Investigate: Check PDB status, healthy replicas, and eviction responses.

Fix: Restore workload redundancy (e.g., scale replicas) or revise the maintenance plan before proceeding.

Senior Insight: Bypassing eviction safeguards can turn routine maintenance into an outage.




```
$ kubectl get pdb
NAME        MIN AVAILABLE   ALLOWED DISRUPTIONS
web-pdb     2               0
$ kubectl drain node-a
error: Cannot evict pod as it would violate
the pod disruption budget "web-pdb".
```

The Service Exists, but Traffic Goes Nowhere

Error 14 — Service Has No Ready Endpoints

Symptom: Service traffic fails even though the Service object exists.

Causes: Incorrect selectors, failing readiness checks, or missing eligible Pods.

Investigate: Check Services, EndpointSlices, labels, and Pod readiness.

Fix: Correct the selection or readiness issue (e.g., labels or probe configuration).

Verification: Eligible endpoints appear and application requests succeed.




```
$ kubectl get svc web-svc
NAME        TYPE        CLUSTER-IP      PORT(S)
web-svc     ClusterIP   10.96.30.10     80/TCP
$ kubectl get endpointslices -l kuberneetes.io/
service-name=web-svc
No resources found
```

Error 15 — Service Traffic Fails Only on Certain Nodes

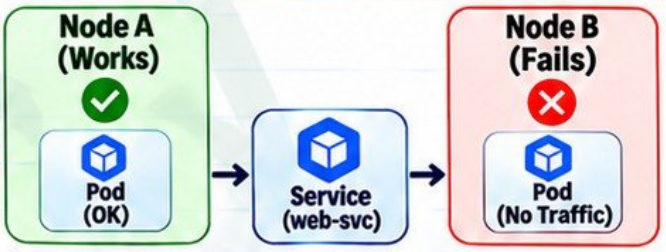
Symptom: Identical workloads behave differently depending on node placement.

Causes: CNI faults, service proxying issues, network policy differences, or node-level networking failures.

Investigate: Compare working and failing nodes, service routing behavior, CNI logs, and packet paths.

Fix: Correct the confirmed node-specific networking fault (e.g., CNI, iptables/IPVS, or policy).

Senior Insight: Node-dependent failures often reveal infrastructure asymmetry rather than application defects.




```
$ kubectl get pods -o wide
 NAME              NODE          STATUS
 web-7f6c          node-a        Running
 web-8d2e          node-b        Running
$ kubectl logs -n kube-system -l k8s-app=cni
... error: failed to program routes on node-b ...
```

Error 16 — Connection Tracking Exhaustion Breaks New Connections

Symptom: New connections fail intermittently while established connections continue working.

Causes: Conntrack table saturation, high connection churn, or excessive retries.

Investigate: Check conntrack utilization, kernel counters, connection rates, and node logs.

Fix: Reduce unnecessary connection churn (e.g., tune retries/keep-alive) and increase capacity based on measured evidence.

Senior Insight: Increasing conntrack limits without fixing connection amplification may only delay recurrence.




```
$ cat /proc/sys/net/netfilter/nf_conntrack_count
262144
$ cat /proc/sys/net/netfilter/nf_conntrack_max
262144
 # count == max  -> conntrack table is saturated
```

DNS Failures That Look Like Application Bugs

Error 17 — CoreDNS Is Healthy but DNS Queries Time Out

Symptom: DNS failures occur despite Running and Ready CoreDNS Pods.

Causes: Upstream resolver problems, packet loss, saturation, or DNS forwarding misconfiguration.

Investigate: Check CoreDNS metrics, query latency, upstream failures, and network reachability.

Fix: Correct the confirmed resolution-path failure (e.g., upstream DNS or forwarding config).

Senior Insight: A healthy DNS process does not guarantee successful DNS resolution.




```
$ kubectl get pods -n kube-system -l k8s-app=kube-dns
NAME                       READY   STATUS    RESTARTS
coredns-6d4f8c7f8c-9k2mw   1/1     Running   0
coredns-6d4f8c7f8c-vt7xz   1/1     Running   0
$ kubectl logs -n kube-system deployment/coredns --tail=5
[ERROR] plugin/errors: 2 example.com. A: read udp
10.96.0.10:56231->8.8.8.8:53: i/o timeout
```

Error 18 — ndots Causes Unexpected External DNS Traffic

Symptom: External hostname lookups generate multiple DNS queries and additional latency.

Causes: Search-domain expansion (e.g., svc.cluster.local) and resolver configuration.

Investigate: Inspect Pod /etc/resolv.conf, query patterns, and DNS metrics (query count and latency).

Fix: Use appropriate fully qualified domain names (FQDN) or adjust resolver settings (e.g., ndots value).

Trade-off: Convenient internal service discovery versus unnecessary DNS amplification.

How ndots Triggers Extra Queries




```
$ kubectl exec -it web-7f9d9c -- cat /etc/resolv.conf
search default.svc.cluster.local svc.cluster.local
       cluster.local
nameserver   10.96.0.10
options      ndots:5
```

Error 19 — DNS Cache Preserves Stale Service Destinations

Symptom: Some clients continue connecting to obsolete addresses after a change.

Causes: Application-level caching, resolver behavior, or unexpectedly long cache retention.

Investigate: Compare DNS answers from different clients and check cache behavior (e.g., TTL local resolver cache).

Fix: Clear or shorten relevant caches (e.g., application resolver or node DNS cache) and verify resolution.

Senior Insight: DNS TTL does not guaranteethat every application refreshes connections immediately.

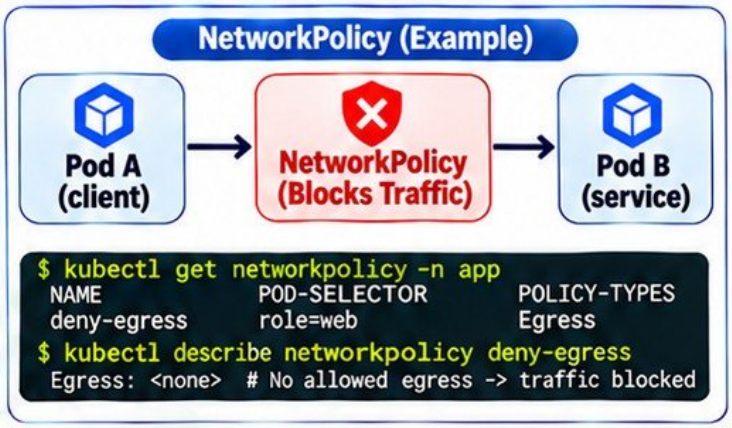



```
# Check DNS from different clients
$ nslookup web-svc.default.svc.cluster.local
 Name:    web-svc.default.svc.cluster.local
 Address: 10.2.1.10   # stale (cached)
$ dig web-svc.default.svc.cluster.local +noall +answer
web-svc.default.svc.cluster.local. 30 IN A  10.2.3.45
```

NetworkPolicy and Ingress Failures That Mislead Engineers

Error 20 NetworkPolicy Silently Blocks Required Traffic

Symptom:	Applications time out after a security policy change.
Causes:	Missing ingress or egress allowances, incorrect selectors, or namespace isolation.
Investigate:	Check effective policies, Pod labels, DNS access, and CNI enforcement.
Fix:	Restore the minimum required connectivity.
Senior Insight:	Policy YAML can be valid while the intended communication path remains blocked.




```
$ kubectl get networkpolicy -n app
NAME           POD-SELECTOR     POLICY-TYPES
deny-egress    role=web         Egress
$ kubectl describe networkpolicy deny-egress
Egress: <none>  # No allowed egress -> traffic blocked
```

Error 21 Ingress Returns 502 or 503 While Pods Are Healthy

Symptom:	Requests fail at the ingress layer despite healthy Pods.
Causes:	Missing endpoints, upstream protocol mismatch, connection failures, or ingress controller config.
Investigate:	Check ingress controller logs, EndpointSlices, upstream connectivity, and HTTP response codes.
Fix:	Correct the failing upstream (e.g., service, port, protocol, or path).
Senior Insight:	A 502 and a 503 can indicate different failure mechanisms, depending on the ingress implementation.




```
$ kubectl logs -n ingress-nginx deploy/ingress-nginx-controller
... 502 Bad Gateway: upstream connect error
... 503 Service Unavailable: no endpoints for service
    "api-svc"
```

Error 22 MTU Mismatch Causes Large Requests to Hang

Symptom:	Small requests succeed while larger payloads stall or time out.
Causes:	Encapsulation overhead, path MTU problems, or blocked ICMP feedback.
Investigate:	Check packet size sensitivity, retransmissions, interface MTU, and packet captures.
Fix:	Correct the path MTU or encapsulation config (e.g., CNI, node MTU, or network device).
Senior Insight:	Packet-size-dependent failures can masquerade as TLS or application timeout problems.




```
$ ip -o link show | grep mtu
eth0     mtu 1500
cni0     mtu 1450
$ ping -M do -s 1472 <pod-ip>
packet too big, MTU = 1450
```

Persistent Volumes That Cannot Follow Their Pods

Error 23 — PVC Remains Pending Despite Available Storage

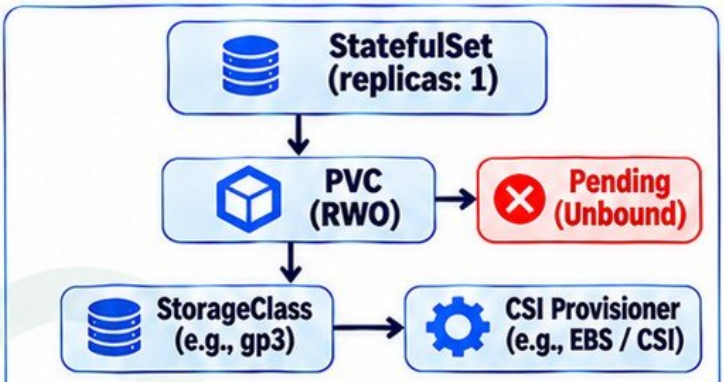
Symptom: A StatefulSet cannot start because its PersistentVolume Claim remains unbound.

Causes: StorageClass mismatch, unavailable provisioner, capacity constraints, or incompatible topology.

Investigate: Check PVC events, StorageClass parameters, CSI provisioner logs, and allowed topologies.

Fix: Correct provisioning constraints or restore compatible storage capacity.

Senior Insight: Storage provisioning and Pod scheduling must agree on topology.




```
$ kubectl get pvc
NAME          STATUS    VOLUME    STORAGECLASS
data-web-0    Pending   <none>    gp3
$ kubectl describe pvc data-web-0
Events:
  Warning  ProvisioningFailed  storageclass "gp3":
  insufficient capacity or invalid topology.
```

Error 24 — Multi-Attach Error After a Node Failure

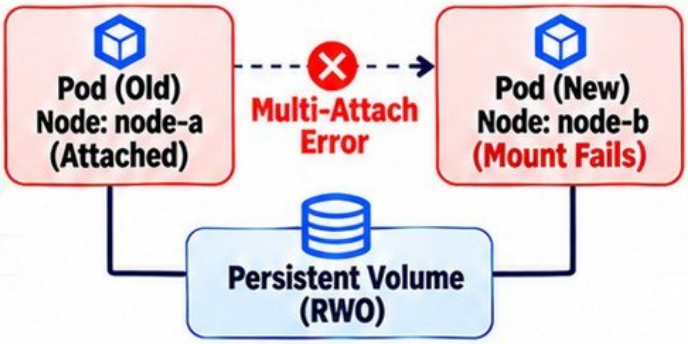
Symptom: A replacement Pod cannot mount its existing volume.

Causes: An existing attachment, delayed detach, or a volume access-mode limitation.

Investigate: Check VolumeAttachment objects, CSI controller logs, cloud attachment state, and old-node status.

Fix: Verify the previous writer is fenced before safely resolving the attachment.

Senior Insight: Forcing a volume detach without confirming write ownership can corrupt stateful data.




```
$ kubectl get volumeattachment
NAME          NODE      ATTACHED
csi-12345     node-a    true
$ kubectl logs -n kube-system -l
app=csi-controller
... Multi-Attach error: volume "pvc-123"
is already attached to node "node-a" ...
```

Storage Performance Failures That Look Like CPU Problems

Error 25 — Storage Latency Causes Application Timeouts

Symptom: Applications become slow despite normal CPU and memory utilization.

Causes: Stroage IOPS saturation, throughput limits, queueing, or backend contention.

Investigate: Volume latency, IOPS, throughput, filesystem behavior, and application traces.

Fix: Correct the measured storage bottleneck or reduce excessive I/O.

Senior Insight: CPU and memory dashboards alone cannot explain storage-bound applications.




```
$ kubectl top pod web-0
NAME      CPU      MEMORY
web-0     120m     256Mi    # looks normal
$ iostat -x 1 5
...
 %util  r/s    w/s   r_await   w_await
 sda    98.5   120   45.2      320.7  # high I/O latency
```

Error 26 — StatefulSet Rollout Stalls During Storage Recovery

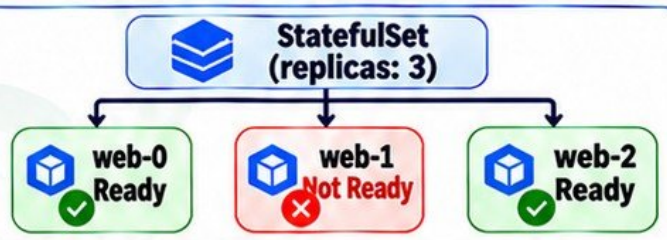
Symptom: A StatefulSet update stops because a replica cannot become Ready.

Causes: Volume attachment delays, failed initialization, storage unavailability, or application recovery requirements.

Investigate: StatefulSet status, Pod events, PVC state, and application recovery logs.

Fix: Recover the blocked replica and its storage dependency before continuing.

Senior Insight: Ordered rollout behavior protects stateful applications but can extend recovery time.




```
$ kubectl get sts web
NAME      READY     CURRENT     UPDATE     AGE
web       2/3       3           3          15m
$ kubectl describe pod web-1
... MountVolume.WaitForAttach: timed out waiting
for volume to be attached ...
```

Error 27 — Volume Mount Succeeds, but the Application Cannot Write

Symptom: The Pod starts but receives permission-denied errors when writing data.

Causes: Filesystem ownership, security contexts, read-only mounts, or unsupported ownership changes.

Investigate: Effective UID/GID, fsGroup, volume permissions, and mount options.

Fix: Correct ownership or security-context configuration without weakening unnecessary permissions.

Senior Insight: A successful mount does not prove that application-level storage access works.




```
$ kubectl exec -it web-0 -- id
uid=1000(app) gid=1000(app) groups=1000(app)
$ kubectl exec -it web-0 -- ls -ld /data
drwx------ 2 root root 4096 Oct 31 09:20 /data
$ kubectl logs web-0 | grep -i permission
ERROR: open /data/file.log: permission denied
```

The API Server Is Available, but the Cluster Is Failing

Error 28 API Server Latency Causes Controller Delays

Symptom: Deployments, scaling operations, and resource updates become unusually slow.

Causes: API request saturation, admission webhook latency, or overloaded control-plane dependencies.

Investigate: API request latency, inflight requests, HTTP 429 responses, and controller work queues.

Fix: Identify and reduce the dominant control-plane bottleneck.

Senior Insight: Control-plane degradation may appear as unrelated workload failures.

Control-Plane Latency Flow




```
$ kubectl get --raw /metrics | grep apiserver
apiserver_request_duration_seconds_bucket ...
apiserver_current_inflight_requests        482
$ kubectl get events -A | grep 429
... TooManyRequests (429): request rate limited ...
```

Error 29 Admission Webhook Failure Blocks Deployments

Symptom: New Pods or resource updates are rejected or delayed while existing workloads continue running.

Causes: Unreachable webhook services, expired certificates, slow webhook responses, or restrictive failure policies.

Investigate: Admission errors, webhook configuration, TLS status, and webhook service health.

Fix: Restore webhook availability or apply an approved emergency policy change.

Trade-off: Fail-closed enforcement protects security but can block recovery operations.

Admission Webhook Request Flow




```
$ kubectl apply -f deploy.yaml
Error from server (InternalError):
error when creating "deploy.yaml":
admission webhook "policy.veriqta.io" failed:
Post "https://webhook-svc.veriqta.svc:443/validate":
context deadline exceeded
$ kubectl get validatingwebhookconfigurations
NAME                FAILURE-POLICY
policy-webhook      Fail
```

etcd and Reconciliation Failures That Threaten Cluster Recovery

Error 30 — etcd Latency Slows Cluster-Wide Operations

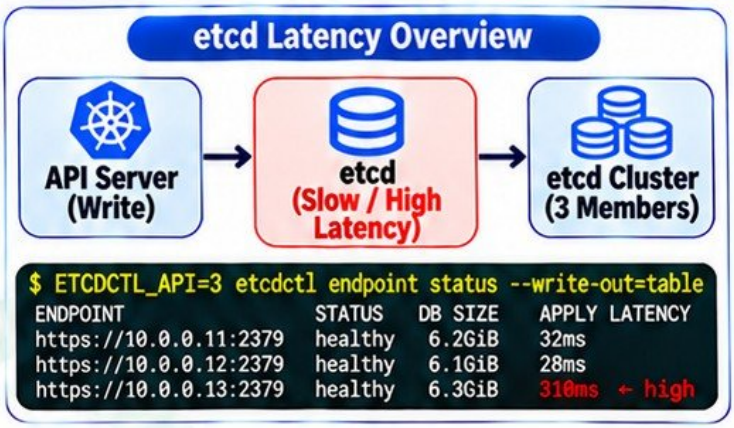
Symptom: Kubernetes API writes become slow, time out, or fail.

Causes: Storage latency, database growth, or etcd member communication problems.

Investigate: etcd request latency, WAL fsync duration, database size, and member health.

Fix: Resolve the confirmed storage or etcd availability issue.

Senior Insight: etcd is a consistency-critical dependency, not a general-purpose database to tune casually.




```
$ ETCDCTL_API=3 etcdctl endpoint status --write-out=table
ENDPOINT                  STATUS   DB SIZE   APPLY LATENCY
https://10.0.0.11:2379    healthy   6.2GiB   32ms
https://10.0.0.12:2379    healthy   6.1GiB   28ms
https://10.0.0.13:2379    healthy   6.3GiB   310ms ← high
```

Error 31 — Controller Reconciliation Repeatedly Reverses Fixes

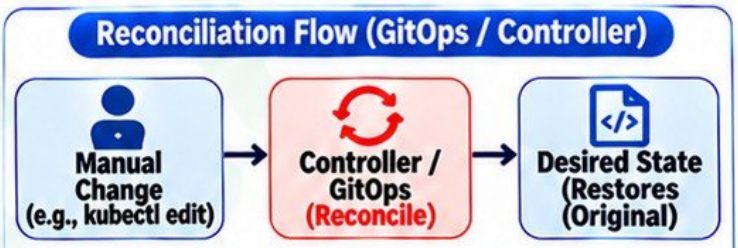
Symptom: Engineers modify a resource, but the change disappears shortly afterward.

Causes: A controller or GitOps reconciler continually restores its declared desired state.

Investigate: Resource ownership, managed fields, controller logs, and GitOps status.

Fix: Update the authoritative configuration rather than repeatedly patching generated resources.

Senior Insight: In declarative systems, the resource you can edit is not necessarily the resource you own.




```
$ kubectl get deploy web -o wide
NAME    READY   UP-TO-DATE   AVAILABLE   AGE
web     3/3     3            3           10m
$ kubectl describe deploy web | grep -i "managed by"
Managed By:    argocd (controller)
```

Error 32 — Kubernetes Finalizers Leave Resources Stuck in Terminating

Symptom: A resource remains in Terminating long after deletion was requested.

Causes: A failed cleanup controller, unavailable external dependency, or incomplete finalization.

Investigate: Finalizers, controller logs, owner references, and external resource state.

Fix: Restore cleanup processing. Remove finalizers manually only after verifying the consequences.

Senior Insight: Force-removing finalizers may leave cloud resources orphaned or external state inconsistent.




```
$ kubectl get pvc data -o jsonpath='{.metadata.finalizers}'
["kubernetes.io/pvc-protection"]
$ kubectl patch pvc data -p '{"metadata":{"finalizers":[]}}'
--type=merge
# Remove finalizer only after confirming cleanup is safe.
```

Kubernetes Security Controls That Break Legitimate Workloads

Error 33 RBAC Denial Breaks an Operator After Deployment

Symptom: An operator runs but cannot reconcile its managed resources.

Causes: Missing verbs, incorrect API groups, namespace scope, or incomplete ClusterRole bindings.

Investigate: API authorization errors, ServiceAccount identity, RBAC bindings, and kubectl auth can-i.

Fix: Grant only the permissions required by the controller.

Senior Insight: RBAC must be validated against actual controller operations, not assumed from installation success.

RBAC Failure Flow




```
$ kubectl auth can-i get pods \\
  --as=system:serviceaccount:operators:my-operator
 no
$ kubectl describe clusterrolebinding my-operator
RoleRef:   ClusterRole/my-operator
Subjects:  ServiceAccount/my-operator (operators)
```

Error 34 Pod Security Admission Rejects Production Pods

Symptom: A workload fails admission after namespace security controls change.

Causes: Privileged containers, forbidden capabilities, host namespace usage, or incompatible security contexts.

Investigate: Admission warnings, namespace Pod Security labels, and Pod specifications.

Fix: Adapt the workload to the required security profile or use a formally governed exception.

Senior Insight: Security policy changes need compatibility testing and rollout controls.

Pod Security Admission (PSA) Flow




```
$ kubectl describe pod web-0
Events:
  Type:     Warning
  Reason:   FailedCreate
  Message:  violates PodSecurity "restricted":
            privileged container is not allowed
$ kubectl get ns prod --show-labels
NAME   LABELS
prod   pod-security.kubernetes.io/enforce=restricted
```

Identity and Secret Failures That Resemble Application Outages

Error 35 — Workload Identity Fails After a ServiceAccount Change

Symptom: Applications suddenly receive cloud API authorization errors.

Causes: Incorrect trust relationships, ServiceAccount configuration, token audience, or cloud role permissions.

Investigate: Projected identity configuration, cloud audit logs, and credential provider errors.

Fix: Restore the correct identity-to-role trust relationship and minimum permissions.

Senior Insight: Kubernetes identity and cloud IAM are separate authorization boundaries.




```
$ kubectl get sa app-sa -n prod -o yaml
...
annotations:
   eks.amaonaws.com/role-arn:  arn:aws:iam::1234:
   role/veriqta-app-role
$ kubectl logs deploy/web -n prod | grep -i "403"
ERROR: AccessDenied: User is not authorized to
perform the requested action (HTTP 403)
```

Error 36 — Secret Rotation Breaks Long-Running Applications

Symptom: Newly started Pods work, but existing Pods continue failing authentication.

Causes: Environment-variable secrets remain unchanged, applications cache credentials, or rotation lacks overlap.

Investigate: Secret update timing, application credential refresh behavior, and authentication errors.

Fix: Implement coordinated credential refresh or safe rollout procedures.

Senior Insight: Updating a Kubernetes Secret does not automaticaliy update environment variables in running containers.




```
$ kubectl get secret app-db -o yaml | grep resourceVersion
 resourceVersion: "45678"
$ kubectl rollout restart deploy/web -n prod
$ kubectl get pods -n prod
 NAME                    STEADY    STATUS
 web-5d7c9f7f6b-2kz9m    1/1       Running
```

Error 37 — Certificate Rotation Breaks Internal Service Communication

Symptom: TLS handshakes fail after certificate renewal.

Causes: Missing trust-chain updates, expired certificates, incorrect SANs, or clients retaining old trust material.

Investigate: Certificate validity, issuer chains, handshake errors, and trust-store versions.

Fix: Restore compatible certificate and trust configuration.

Senior Insight: Certificate rotation is a distributed compatibility event, not simply a file replacement.




```
$ openssl s_client -connect svc-b.prod.svc:443 -showcerts
...
verify error:num=10:certificate has expired
$ kubectl get secret svc-b-tls -n prod -o yaml | grep notAfter
 notAfter: "2026-10-31T23:59:59Z"
$ kubectl describe certificate svc-b-cert -n prod
 Status:  Ready: False, Reason: Expired
```

Autoscaling Creates More Problems Than It Solves

Error 38 — HPA Continuously Scales Up and Down

Symptom: Replica counts fluctuate rapidly without stable application performance.

Causes: Noisy metrics, inappropriate utilization targets, workload startup delays, or insufficient stabilization.

Investigate: HPA conditions, scaling events, metrics history, and application response time.

Fix: Correct scaling signals and configure appropriate stabilization behavior.

Senior Insight: Autoscaling must account for workload response time, not just metric thresholds.

HPA Thrashing (Scale Up and Down)




```
$ kubectl get hpa web -n prod
NAME    REFERENCE      TARGETS    MIN   MAX   REPLICAS
web     deploy/web     72%/70%    2     10    5
$ kubectl describe hpa web -n prod | grep -i "scale"
Events:
  Normal   SuccessfulRescale   Scaled up replica set to 5
  Normal   SuccessfulRescale   Scaled down replica set to 2
```

Error 39 — Scaling Pods Makes Database Performance Worse

Symptom: Adding application replicas increases latency and database failures.

Causes: Connection pool multiplication, database contention, and unbounded downstream concurrency.

Investigate: Replica counts, database connections, lock waits, and query latency.

Fix: Limit connection pressure and align application concurrency with downstream capacity.

Senior Insight: Horizontal scaling can amplify a bottleneck instead of removing it.

More Replicas, Higher DB Contention




```
$ kubectl get deploy web -n prod
NAME    READY   UP-TO-DATE   AVAILABLE   REPLICAS
web     10/10   10           10          10
$ kubectl exec -it db-0 -- psql -c \
"select count(*) from pg_stat_activity;"
 count
-------
   312   # connections increased after scaling
```

Production Capacity Failures Hidden by Healthy Dashboards

Error 40 — Node Autoscaling Cannot Keep Up With Sudden Demand

Symptom: Application Pods remain Pending during a traffic spike.

Causes: Instance provisioning delay, insufficient cloud capacity, image pull duration, or scheduling constraints.

Investigate: Pending duration, node provisioning events, image pull latency, and capacity availability.

Fix: Maintain appropriate headroom or use workload-aware capacity strategies.

Senior Insight: Reactive autoscaling cannot eliminate the time required to provision and initialize capacity.

Autoscaling Provisioning Flow




```
$ kubectl get pods -n prod
NAME          READY   STATUS    AGE
api-7f8d9     0/1     Pending   2m18s
api-9c2b1     0/1     Pending   2m17s
$ kubectl get events -n prod | grep -i "scal"
Normal   TriggeredScaleUp   ClusterAutoscaler
Normal   Provisioning       node (in progress)
```

Error 41 — Metrics Cardinality Overloads the Monitoring Platform

Symptom: Prometheus consumes excessive memory or fails to process metrics reliably.

Causes: Unbounded labels, per-request identifiers, excessive series creation, or retention pressure.

Investigate: Active series counts, high-cardinality labels, ingestion rate, and TSDB metrics.

Fix: Remove unbounded label dimensions and apply appropriate collection limits.

Senior Insight: Observability infrastructure has its own capacity and reliability limits.

Metrics Cardinality Problem




```
$ curl -s http://prometheus:9090/api/v1/status/tsdb \
| grep -E "numSeries|headSeries"
   "numSeries": 4827316,
   "headSeries": 4819023,
# Too many series → high memory and slower queries
```

Error 42 — Logging Backpressure Consumes Node Storage

Symptom: Node disk usage grows rapidly during an application incident.

Causes: Excessive log generation, delayed log collection, retention problems, or unbounded temporary logs.

Investigate: Container log sizes, node filesystem metrics, collector health, and log rates.

Fix: Restore log collection, control log volume, and enforce retention.

Senior Insight: An incident can become a second incident when diagnostic telemetry overwhelms infrastructure.

Logging Backpressure Flow




```
$ kubectl exec node-1 -- df -h /var/log
Filesystem      Size  Used Avail Use% Mounted on
/dev/nvme0n1p1  100G   92G    6G  94% /var/log
$ kubectl logs fluent-bit -n logging
...[warn] output queue is full, backing off...
```

Kubernetes Deployments That Look Successful but Are Unsafe

Error 43 — Rolling Update Causes Temporary Capacity Collapse

Symptom: Request failures increase during a deployment despite healthy replicas beforehand.

Causes: Incorrect maxUnavailable, insufficient capacity, premature readiness, or slow application warm-up.

Investigate: Deployment events, rollout settings, ready endpoints, and request latency.

Fix: Correct rollout capacity and readiness behavior.

Senior Insight: Deployment strategy must account for actual serving capacity, not only replica counts.

Rolling Update Capacity Flow




```
$ kubectl get deploy web -o wide
 NAME      READY     UP-TO-DATE      AVAILABLE
 web       2/3       3               2
$ kubectl describe deploy web | grep -E \
 "maxUnavailable|maxSurge"
  MaxUnavailable:    1
  MaxSurge:          1
```

Error 44 — Rollback Restores Pods but Not Application Compatibility

Symptom: Previous application images start successfully but fail against changed database or external state.

Causes: Incompatible schema migrations, altered message formats, or irreversible side effects.

Investigate: Release history, migration records, database state, and application compatibility.

Fix: Use a compatible forward fix or a verified recovery strategy.

Senior Insight: Kubernetes rollout undo reverses a workload template, not the entire production system.

Rollback vs System State




```
$ kubectl rollout history deploy web
REVISION    IMAGE              CHANGE-CAUSE
1           web:v1.0.0         initial release
2           web:v2.0.0         add new db fields
$ kubectl rollout undo deploy web --to-revision=1
deployment.apps/web rolled back to revision 1
# Pods run on v1, but db schema is still v2
  → application may still fail.
```

GitOps and Deployment Controllers Fighting Over Production

Error 45 — Argo CD Reports OutOfSync After a Successful Deployment

Symptom: A deployed application works, but GitOps continuously reports configuration drift.

Causes: Mutating admission webhooks, defaulted fields, multiple field managers, or generated configuration.

Investigate: Desired-versus-live diffs, managed fields, and reconciliation history.

Fix: Correct configuration ownership or narrowly define legitimate diff exceptions.

Senior Insight: Ignoring all drift hides dangerous changes. Ignoring only understoad differences preserves control.

Argo CD Drift Flow




```
$ argocd app get web
NAME        SYNC STATUS    HEALTH      REVISION
web         OutOfSync      Healthy     a1b2c3d
$ argocd app diff web
- spec.replicas: 2      (desired)
+ spec.replicas: 3      (live)
# Drift caused by defaulting or another controller
```

Error 46 — Two Controllers Continuously Overwrite the Same Resource

Symptom: Configuration repeatedly changes between competing values.

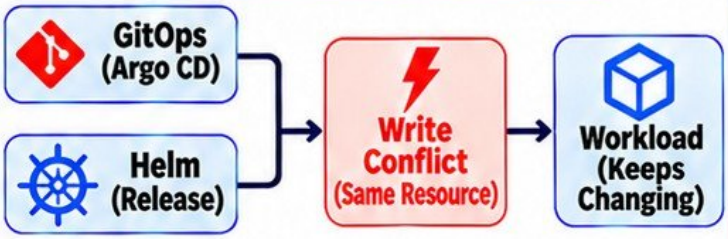
Causes: Multiple GitOps applications, Helm ownership conflicts, or overlapping controllers.

Investigate: Resource ownership, field managers, reconciliation events, and deployment definitions.

Fix: Establish a single authoritative owner for conflicting configuration fields.

Senior Insight: Automation without clear ownership can create persistent configuration instability.

Controller Conflict Flow




```
$ kubectl get deployment web -o yaml | grep managedFields
managedFields:
  - manager: argocd
    operation: Apply
  - manager: helm
    operation: Update
# Conflicting controllers are both managing the same fields
```

The Incident That Spread Across the Entire Cluster

Error 47 — A DaemonSet Rollout Breaks Networking Across Nodes

Symptom: Pods lose connectivity as a networking DaemonSet update progresses.

Causes: Incompatible CNI configuration, failed agent startup, or an unsafe rollout.

Investigate: DaemonSet revision history, node-level agent health, CNI logs, and connectivity by node.

Fix: Stop the faulty rollout and restore a verified compatible networking configuration.

Senior Insight: Infrastructure DaemonSets can create correlated failures across otherwise independent workloads.

Networking DaemonSet Failure Flow




```
$ kubectl rollout status ds/cni -n kube-system
Waiting for DaemonSet "cni" rollout to finish...
$ kubectl get pods -n kube-system -o wide
NAME        READY   STATUS             NODE
cni-8f7d9   0/1     CrashLoopBackOff   node-1
cni-k2m4p   0/1     CrashLoopBackOff   node-2
```

Error 48 — A Cluster Upgrade Breaks Critical Controllers

Symptom: Workloads continue running, but deployments, autoscaling, or custom resource reconciliation fail after an upgrade.

Causes: Removed APIs, incompatible controllers, webhook failures, or version skew.

Investigate: API compatibility, controller logs, deprecated API usage, and upgrade history.

Fix: Restore compatible components using a supported recovery path.

Senior Insight: Control-plane compatibility must be verified before and after upgrading production clusters.

Cluster Upgrade Compatibility Flow




```
$ kubectl get apiservices | grep False
v1beta1.metrics.k8s.io                 False
v1beta1.custom.metrics.k8s.io          False
$ kubectl logs deploy/custom-controller -n platform
error: no matches for kind "Widget" in version
"example.com/v1beta1   (removed or incompatible API)
```

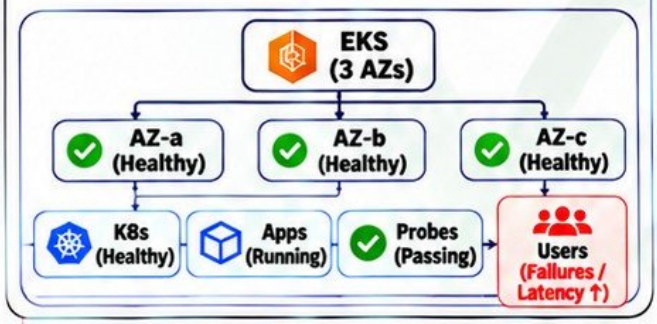
The Kubernetes Cluster Is Running, but the Business Is Offline

Error 49 — Multiple Healthy Components Conceal a Shared Dependency Failure

Production Scenario:

An EKS cluster spans three Availability Zones. Kubernetes reports most nodes Ready, core services are running, and application Pods pass their liveness checks.

However, users experience intermittent failures, increased latency, and unsuccessful transactions.



Investigation:

- Determine whether failures correlate with specific nodes, zones, or request paths.
- Compare ingress error rates, EndpointSlices, DNS latency, and database connectivity.
- Identify whether retries and autoscaling are amplifying a shared dependency failure.
- Establish which services can safely degrade and which must reject requests.

Mitigation:

- Mitigate the confirmed failure while preserving transaction correctness.

Senior Challenge:

- Demonstrate why healthy infrastructure components do not necessarily produce a healthy business service.

Error 50 — Emergency Recovery Fixes the Outage but Creates a Second Incident

Production Scenario:

During a cluster-wide incident, an Dngineer proposes disabling admission webhooks, removing PodDisruptionBudgets, relaxing NetworkPolicies, and restarting critical workloads simultaneously.

These changes may appear to accelerate recovery, but they also introduce security, availability, and data-integrity risks.

- ✗ Disable Webhooks
- ✗ Remove PDBs
- ✗ Relax NetPolicies
- ✗ Restart Workloads

Fast Recovery (Seems Good)

But Higher Risk:

- Security Gaps
- Service Disruption
- Data Inconsistency

Investigation and Decision:

- Identify which controls are actually contributing to the outage.
- Establish the smallest reversible mitigation.
- Determine which emergency changes require additional approval.
- Avoid simultaneous changes that destroy root-cause evidence.
- Verify application health, transaction correctness, and cluster stability.
- Restore temporary security exceptions and reconcile configuration drift.
- Document the incident timeline and prevention measures.

Final Senior Engineer Challenge:

- Recover production safely without weakening the entire cluster or introducing an untraceable second failure.